

Рисование графиков на Python

М. А. Ложников

31 октября 2019 г.

Ревизия: 0

1 Введение в Python

1.1 Циклы, операторы условного перехода и ввод/вывод

1.1.1 Пример 1

```
1 # Это комментарий
2
3 """
4 Это многострочный
5 комментарий
6 """
7
8 '''
9 Это тоже
10 многострочный комментарий
11 '''
12
13 i = 0      # Объявление переменной
14 n = 10     # Тип данных определяется автоматически
15
16 n = "abc"  # Тип данных может измениться при новом присваивании (теперь это строка).
17
18 print(type(n)) # Выведет информацию о типе данных переменной n
19
20 # Это обычный цикл while. Тело цикла пишется с отступом. Ширина отступа должна быть
21 # одинаковой. Обычно используют 4 пробела. Табуляцию использовать не стоит,
22 # поскольку она отображается по-разному в разных редакторах. Python 3 запрещает
23 # смешивать табуляции и пробелы.
24 n = 10
25 while i < n:
26     print(i) # вывод на экран
27     i += 1
28
29 # Арифметические операторы: +, -, *, /, //, %, **, +=, -=, *=, /=, //=, %=, **=
30 # Логические операторы: <, >, <=, >=, ==, !=
31 # Отрицание: not
32 # Логическое и/или: and/or
33 # Битовые операции: |, &, ^, ~, <<, >>
34
```

```

35 # Деление / всегда вещественное. Операция // используется для целочисленного деления.
36
37 # Примеры:
38 a = 15
39 b = -5
40 c = a ** 3 # Возведение числа a в степень 3
41
42 # Логические операторы ленивые, они не вычисляются полностью. Если значение выражения
43 # уже известно, то операнды могут дальше не вычисляться.
44 s1 = "abc"
45 s2 = "def"
46 s = s1 or s2 # s = "abc" (Отлично от None и пустой строки, значит, истина.)
47 # Результат логического выражения не приводится к типу bool, значениями которого
48 # являются True и False.
49
50 # Тело оператора условного перехода тоже пишется с отступом. Слово pass пишут тогда,
51 # когда хотят оставить тело цикла, условного перехода, функции, класса пустым.
52 if not (a > 0):
53     pass
54 if a > 0 and b < 0:
55     pass
56
57 c = None # Это пустое значение, аналог NULL в C++.
58
59 # Операторы сравнения == и != можно переопределить, поэтому сравнение с None
60 # следует делать следующим образом:
61 if c is None:
62     print('c is none')
63
64 if c is not None:
65     print('c is not none')

```

1.1.2 Пример 2

```

1 i = 0
2 n = 10
3
4 while i < n:
5     if i < 5:
6         print('i < 5 and i = %d' % (i))
7     elif i == 5:
8         print('i = 5')
9     else:
10        print('i > 5 and i = %d' % (i))
11    i += 1
12
13
14 # Приведение типов данных
15 a = 3.5 # Для хранения используется тип данных float
16 i = int(a) # Преобразуем дробное число в целое
17
18 s = '123.45' # Строка

```

```
19 d = float(s) # Преобразуем строку в число
20 t = str(d) # Преобразуем число в строку
21 g = repr(d) # То же самое, только без округлений, способствующих читабельности
22
23 c_num = 1 + 2j # Это комплексное число
24 print(type(c_num))
25 print(c_num.real) # Действительная часть
26 print(c_num.imag) # Мнимая часть
```

1.1.3 Пример 3

```
1 # Так импортируются модули. Это обычные файлы на Python, которые содержат реализации
2 # функций и классов, а также объявления констант.
3 import math
4
5 # Функция input() выводит приглашение для ввода в стандартный поток вывода и
6 # считывает строку из стандартного потока ввода.
7 n = int(input('Enter n-> '))
8
9 # Такой цикл проходит по всем целым значениям от 0 до n-1.
10 for a in range(0, n):
11     print(a / math.pi) # Константа pi из модуля math --- это число Пи.
```

1.2 Функции

```
1 # Объявим функцию, которая складывает два своих аргумента. Тело функции пишется
2 # с отступом.
3 def sum_func(a, b):
4     return a + b
5
6 # Вызов функции
7 n = sum_func(2, 3) # n = 5
8 s = sum_func('abc', 'def') # s = 'abcdef'
9
10 # В Python при вызове функции можно передавать какому аргументу что присваивается.
11 # Это бывает полезно, например, в тех случаях, когда функция содержит много аргументов,
12 # значение которых задано по умолчанию.
13 c = sum_func(a=2, b=3) # c = 5
```

1.3 Строки

```
1 # Строка задаётся либо внутри двойных кавычек либо внутри одинарных
2 # Строки состоят из unicode символов.
3 a = "Hello World!"
4 b = a[4] # b = "o"
5
6 c = "34"
7 d = "56"
8 e = c + d # e = "3456" Это - конкатенация строк
```

```
9
10 g = a[0:5] # g = "Hello". Операция срезки всегда создаёт новую строку.
11 h = a[6:11] # h = "World"
```

1.3.1 Форматирование строк

```
1 num = 3
2 pi = 3.14159
3
4 # Так формируются строки. Формат записывается в стиле функции printf() языка C,
5 # затем после операции % идёт кортеж со значениями, которые нужно подставить.
6 s = "Num = %d and Pi = %.3f" % (num, pi)
7
8 # Ещё можно использовать метод format() класса str.
9 s = "Num = {} and Pi = {}".format(num, pi)
10
11 # Можно указать формат вывода в скобках после двоеточия.
12 s = "Num = {:05d} and Pi = {:.3f}".format(num, pi)
13
14 # Можно передать именованные аргументы:
15 s = "Num = {integer_arg} and Pi = {float_arg}".format(
16     integer_arg=num, float_arg=pi)
17
18 # Можно добавить формат вывода
19 s = "Num = {integer_arg:05d} and Pi = {float_arg:.3f}".format(
20     integer_arg=num, float_arg=pi)
21
22 # В Python 3.6 появились F-строки.
23 s = f"Num = {num} and Pi = {pi}"
24
25 # Можно с форматом
26 s = f"Num = {num: 05d} and Pi = {pi: .3f}"
```

2 Работа с массивами с помощью библиотеки numpy

2.1 Импорт библиотеки

Обычно модуль `numpy` импортируют так:

```
1 import numpy as np
```

В этом случае к функциям из этого модуля следует добавлять префикс `np` с точкой, например, `np.array()` и т. д.

2.2 Создание и индексация массива

```
1 a = np.array([1, 4, 5, 8], float) # Создаём массив из списка.
2 # Тип данных (второй параметр) можно не указывать.
3
```

```
4 # Массивы индексируются точно так же, как и списки
5 b = a[:2] # b = [1., 4.]
6 c = a[3] # c = 8.;
7 a[0] = 5 # Меняем самый первый элемент массива a на 5
```

Кроме того, массив можно загрузить из текстового файла.

Если массив одномерный, то его элементы должны располагаться в одной строке текстового файла через пробел либо по одному элементу в строке.

Двумерный массив (матрица) размера $N \times M$ должен быть задан следующим образом: в i -ой строке текстового файла, $i = 1, \dots, N$, должны располагаться ровно M чисел через пробел.

```
1 a = np.loadtxt('array.txt')
```

и сохранить в текстовый файл

```
1 np.savetxt('array.txt', a)
```

Существуют также функции, позволяющие загружать и сохранять массив в бинарном формате, они работают намного быстрее.

Массивы могут быть многомерными, вот пример матрицы (двумерный массив).

```
1 # Матрицу можно создать из двумерного списка
2 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
3 >>> a
4 array([[ 1.,  2.,  3.],
5         [ 4.,  5.,  6.]])
6 >>> a[0, 0] # Матрица индексируется номером строки и номером столбца через запятую
7 1.0 # в одних квадратных скобках. Строки и столбцы ну
8 >>> a[0, 1]
9 2.0
10 # Создание массива равномерно распределённых точек
11 >>> a = np.arange(0, 1, 0.1) # С заданным шагом
12 >>> a
13 array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])
14 >>> a = np.linspace(0, 1, num=10) # С заданным количеством точек
15 >>> a
16 array([0. , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
17         0.55555556, 0.66666667, 0.77777778, 0.88888889, 1. ])

```

В примерах этого раздела будем считать, что весь код вводится в интерпретатор Python в интерактивном режиме, символы `>>>` означают приглашение для ввода, выводимое интерпретатором, после чего следует введённая пользователем команда. Команда в результате своей работы может вывести что-либо на экран (такие строки не содержат символов `>>>` в начале). Для читабельности некоторые команды отделены друг от друга пустыми строками.

Срезки используются точно так же как и со списками и берутся отдельно по разным измерениям.

```
1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
2 >>> a[1, :]
3 array([ 4.,  5.,  6.]) # Последняя строка матрицы
4 >>> a[:, 2] # Использование : указывает на то, что нужно взять всё измерение целиком

```

```

5 array([ 3.,  6.])      # Последний столбец матрицы (одномерный массив)
6
7 >>> a[-1, -2:]        # Если индексы отрицательные, то это значит, что эти числа следует
8                        # вычесть из размера массива по соответствующему измерению
9 array([ 5.,  6.])      # На выходе одномерный массив т.к. указана только одна строка.
10
11 >>> a[-1:, -2:]        # В этом примере на выходе будет матрица даже в том случае,
12 array([[ 5.,  6.]])    # если она состоит из одной строки (столбца) поскольку по обоим
13                        # направлениям указаны срезы.

```

Размеры матрицы хранятся в кортеже **shape**

```

1 >>> a.shape
2 (2, 3)

```

Функция **len** возвращает размер массива по первой координате

```

1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float) # Матрица 2*3
2 >>> len(a)
3 2

```

Для проверки принадлежности массиву используется ключевое слово **in**. С массивами чисел с плавающей точкой лучше не использовать.

```

1 >>> a = np.array([[1, 2, 3], [4, 5, 6]], float)
2 >>> 2 in a
3 True
4 >>> 0 in a
5 False

```

Элементы массива можно перегруппировать, например,

```

1 >>> a = np.array(range(10), float)
2 >>> a
3 array([ 0.,  1.,  2.,  3.,  4.,  5.,  6.,  7.,  8.,  9.])
4 >>> b = a.reshape((5, 2)) # Функция принимает кортеж с размерностями нового массива
5 >>> b                      # Важно: массивы a и b ссылаются на одни и те же данные.
6 array([[ 0.,  1.],
7        [ 2.,  3.],
8        [ 4.,  5.],
9        [ 6.,  7.],
10       [ 8.,  9.]])
11 >>> b.shape
12 (5, 2)

```

Функция **reshape()** возвращает массив указанной формы, полученный из другого массива, при этом она не модифицирует старый. Однако, оба массива будут ссылаться на одни и те же данные, то есть при изменении элементов одного массива, поменяются и соответствующие элементы второго. Для полного (глубокого) копирования массива следует использовать функцию **copy()**. Иначе python может просто создать ссылку.

2.3 Операции с массивами

```
1 >>> a = np.array([1,2,3], float)
2 >>> b = np.array([5,2,6], float)
3 >>> a + b
4 array([6., 4., 9.])
5 >>> a - b
6 array([-4., 0., -3.])
7 >>> a * b
8 array([5., 4., 18.])
9 >>> b / a
10 array([5., 1., 2.])
11 >>> a % b
12 array([1., 0., 3.])
13 >>> b ** a
14 array([5., 4., 216.])
15 # Обратите внимание, что умножение - поэлементное
16 >>> a = np.array([[1,2], [3,4]], float)
17 >>> b = np.array([[2,0], [1,3]], float)
18 >>> a * b
19 array([[2., 0.], [3., 12.]])
```

К массивам можно применять поэлементно функции `abs`, `sign`, `sqrt`, `log`, `log10`, `exp`, `sin`, `cos`, `tan`, `arcsin`, `arccos`, `arctan`, `sinh`, `cosh`, `tanh`, `arcsinh`, `arccosh`, и `arctanh`, например,

```
1 >>> a = np.array([1, 4, 9], float)
2 >>> np.sqrt(a)
3 array([ 1.,  2.,  3.])
```

Массивы можно округлять поэлементно

```
1 >>> a = np.array([1.1, 1.5, 1.9], float)
2 >>> np.floor(a)
3 array([ 1.,  1.,  1.])
4 >>> np.ceil(a)
5 array([ 2.,  2.,  2.])
6 >>> np rint(a)
7 array([ 1.,  2.,  2.])
```

Кроме того

```
1 >>> np.pi
2 3.1415926535897931
3 >>> np.e
4 2.7182818284590451
```

2.4 Базовые операции с массивами

2.4.1 Сумма и произведение всех эл-тов

```
1 >>> a = np.array([2, 4, 3], float)
2 >>> a.sum()
3 9.0
4 >>> a.prod()
5 24.0
6 >>> np.sum(a)
7 9.0
8 >>> np.prod(a)
9 24.0
```

2.4.2 Среднее и вариация

```
1 >>> a = np.array([2, 1, 9], float)
2 >>> a.mean()
3 4.0
4 >>>
5 a.var()
6 12.666666666666666
7 >>> a.std()
8 3.5590260840104371
```

2.4.3 Минимум и Максимум

```
1 >>> a = np.array([2, 1, 9], float)
2 >>> a.min()
3 1.0
4 >>> a.max()
5 9.0
6 >>> a = np.array([2, 1, 9], float)
7 >>> a.argmin()
8 1
9 >>> a.argmax()
10 2
```

2.4.4 Применение базовых операций вдоль выбранной оси

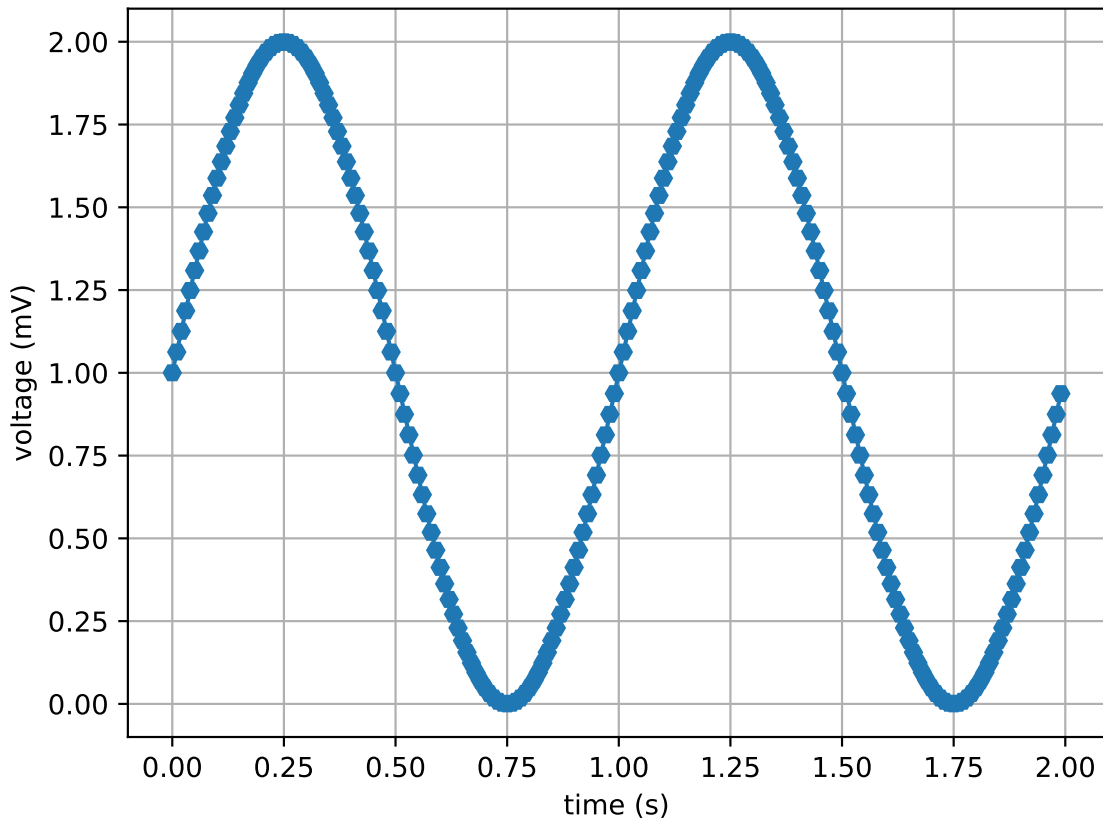
```
1 >>> a = np.array([[0, 2], [3, -1], [3, 5]], float)
2 >>> a.mean(axis=0)
3 array([ 2.,  2.])
4 >>> a.mean(axis=1)
5 array([ 1.,  1.,  4.])
6 >>> a.min(axis=1)
7 array([ 0., -1.,  3.])
8 >>> a.max(axis=0)
9 array([ 3.,  5.])
```

3 Библиотека matplotlib в примерах

3.1 Простейший пример

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Создаём массивы
5 t = np.arange(0.0, 2.0, 0.01)
6 s = 1 + np.sin(2*np.pi*t)
7
8 # Рисуем график
9 plt.plot(t, s)
10
11 # Подписываем оси
12 plt.xlabel('time (s)')
13 plt.ylabel('voltage (mV)')
14
15 # Заголовок рисунка
16 plt.title('About as simple as it gets, folks')
17
18 # Рисовать или нет координатную сетку
19 plt.grid(True)
20
21 # Сохранить в файл (PNG, EPS или PDF)
22 plt.savefig("test.png")
23
24 # Сохраняем в файл с заданным числом пикселей на дюйм и минимальными рамками
25 plt.savefig("test2.png", dpi = 300, bbox_inches='tight')
26
27 # Сохраняем в векторном формате EPS
28 plt.savefig("simple_plot.eps", bbox_inches='tight')
29
30 # Сохраняем в векторном формате PDF
31 plt.savefig("simple_plot.pdf", bbox_inches='tight')
32
33 # Вывести на экран
34 plt.show()
```

About as simple as it gets, folks



Полезные параметры `plt.plot`

- Стилль линии `linestyle` или `ls`

['-', '--', '-.', ':', 'steps' | ...]

- Толщина линии `linewidth` или `lw`.

- `marker`

['.', ',', 'o', 'v', '^', '<', '>', '1', '2', '3', '4', '8', 's', 'p', '*',
'h', 'H', '+', 'x', 'D', 'd', '|', '_', 'P', 'X', 0, 1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, ' ', '']

- `color` — цвет.

Например,

```
1 plt.plot(t, s, lw = 4.5, ls = '-', marker = ':', color='r')  
2 # или color = b,g, и т.д. Или color=(0,1,1) - в формате RGB.
```

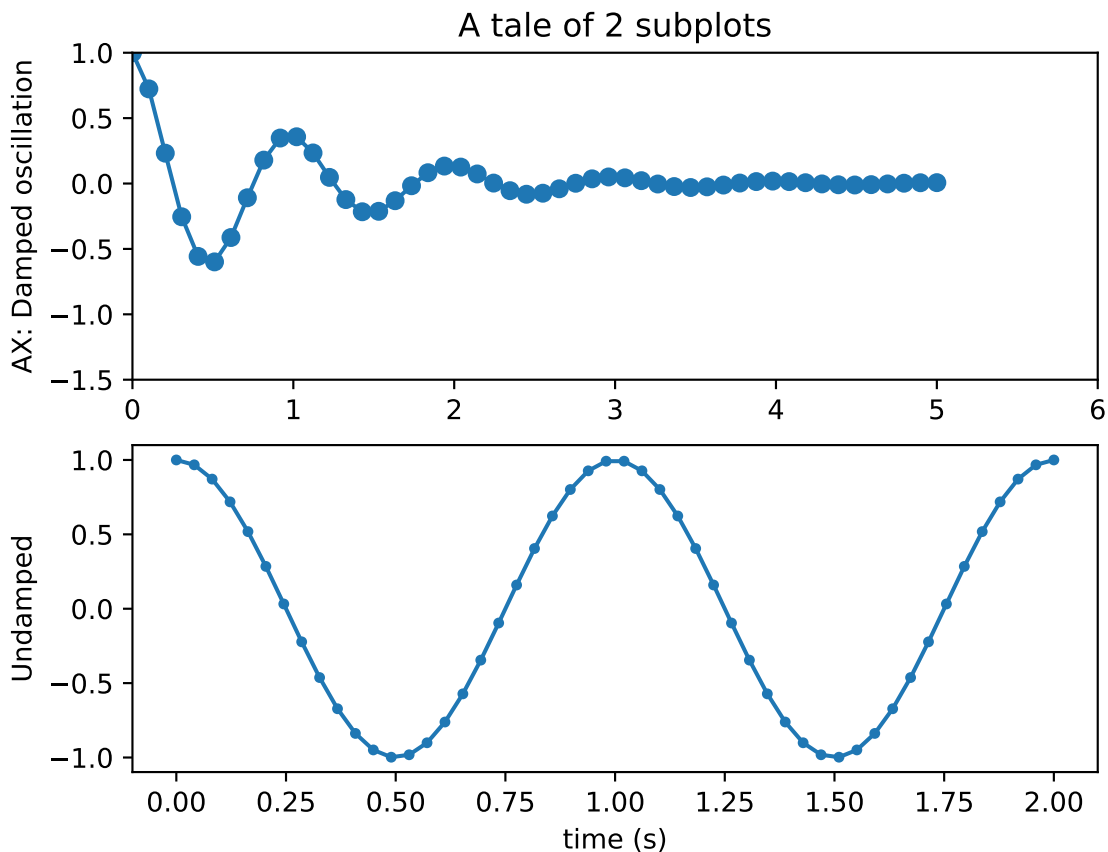
3.1.1 Несколько графиков на одном рисунке

```
1 import numpy as np  
2 import matplotlib.pyplot as plt  
3  
4 # Создаём равномерно распределённые массивы точек
```

```

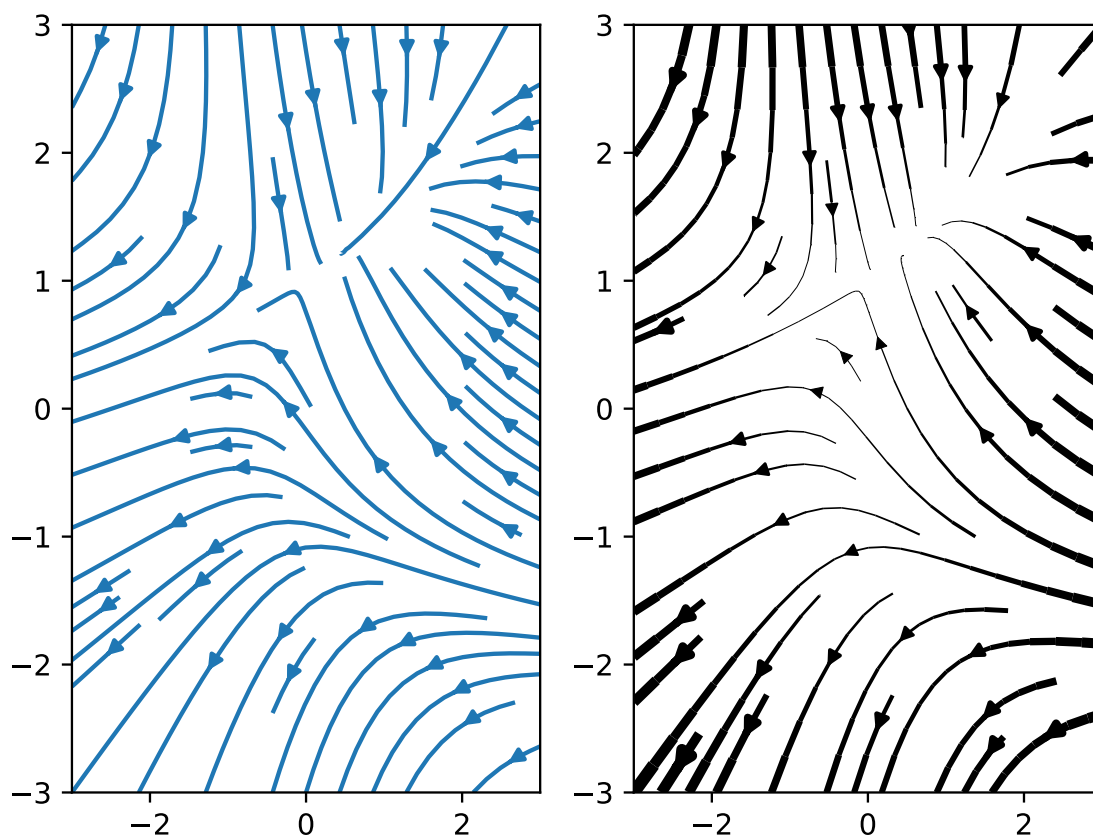
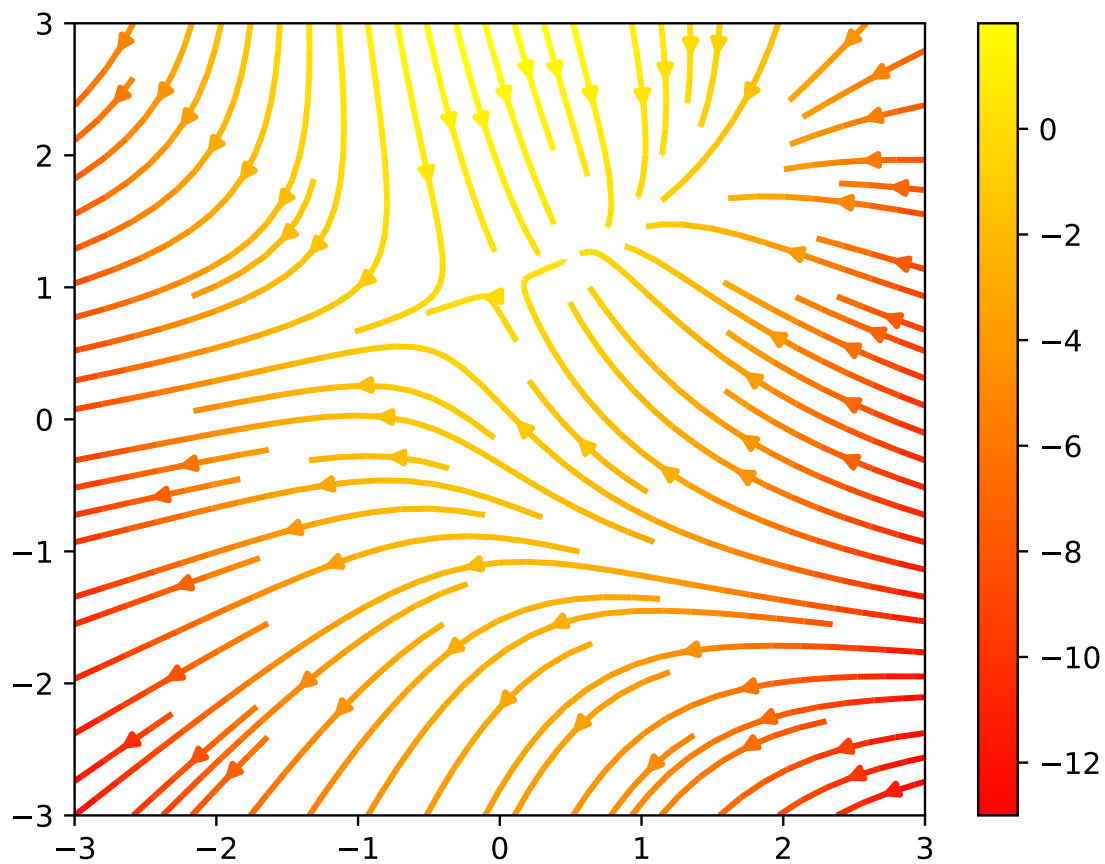
5 # в заданных промежутках
6 x1 = np.linspace(0.0, 5.0, num = 50)
7 x2 = np.linspace(0.0, 2.0)
8
9 y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
10 y2 = np.cos(2 * np.pi * x2)
11
12 # Выбираем первый график
13 # Первый параметр - общее количество графиков
14 # Второй и третий параметры отвечают за их
15 # пространственное расположение друг относительно друга
16 ax = plt.subplot(2, 1, 1)
17 plt.plot(x1, y1, 'o-')
18 plt.title('A tale of 2 subplots')
19 ax.set_ylabel('AX: Damped oscillation')
20 ax.set_xlim(0, 6)
21 ax.set_ylim(-1.5, 1)
22
23 plt.subplot(2, 1, 2)
24 plt.plot(x2, y2, '-.')
25 plt.xlabel('time (s)')
26 plt.ylabel('Undamped')
27
28 plt.savefig("subplot_demo1.pdf", bbox_inches='tight')
29
30 plt.show()

```



3.2 Streamlines - Трубки тока векторного поля

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Создаём двумерную сетку 100x100 на области [-3, 3] * [-3, 3]
5 x = np.linspace(-3, 3, num=100)
6 y = np.linspace(-3, 3, num=100)
7 X, Y = np.meshgrid(x, y)
8
9 # Задаём компоненты функции скорости на сетке
10 U = -1 - X**2 + Y
11 V = 1 + X - Y**2
12
13 # Тогда модуль скорости равен
14 speed = np.sqrt(U*U + V*V)
15
16 # первый рисунок
17 # plt.subplots() возвращает кортеж
18 # классов figure и axis, которые описывают рисунок
19 # и координатные оси соответственно
20 fig0, ax0 = plt.subplots()
21
22 # Первый рисунок
23 strm = ax0.streamplot(X, Y, U, V, color=U, linewidth=2, cmap=plt.cm.autumn)
24
25 # Добавляем цветовой индикатор
26 fig0.colorbar(strm.lines)
27
28 # Сохраняем первый рисунок
29 fig0.savefig("streamplot.pdf", bbox_inches='tight')
30
31
32 # Второй и третий графики в отдельном рисунке
33 # ncols - число столбцов
34 fig1, (ax1, ax2) = plt.subplots(ncols=2)
35 ax1.streamplot(X, Y, U, V, density=[0.5, 1])
36
37 lw = 5*speed / speed.max()
38 ax2.streamplot(X, Y, U, V, density=0.6, color='k', linewidth=lw)
39
40 # Сохраняем второй рисунок
41 fig1.savefig("streamplot1.pdf", bbox_inches='tight')
42
43
44 plt.show()
```



3.3 Контурь

```

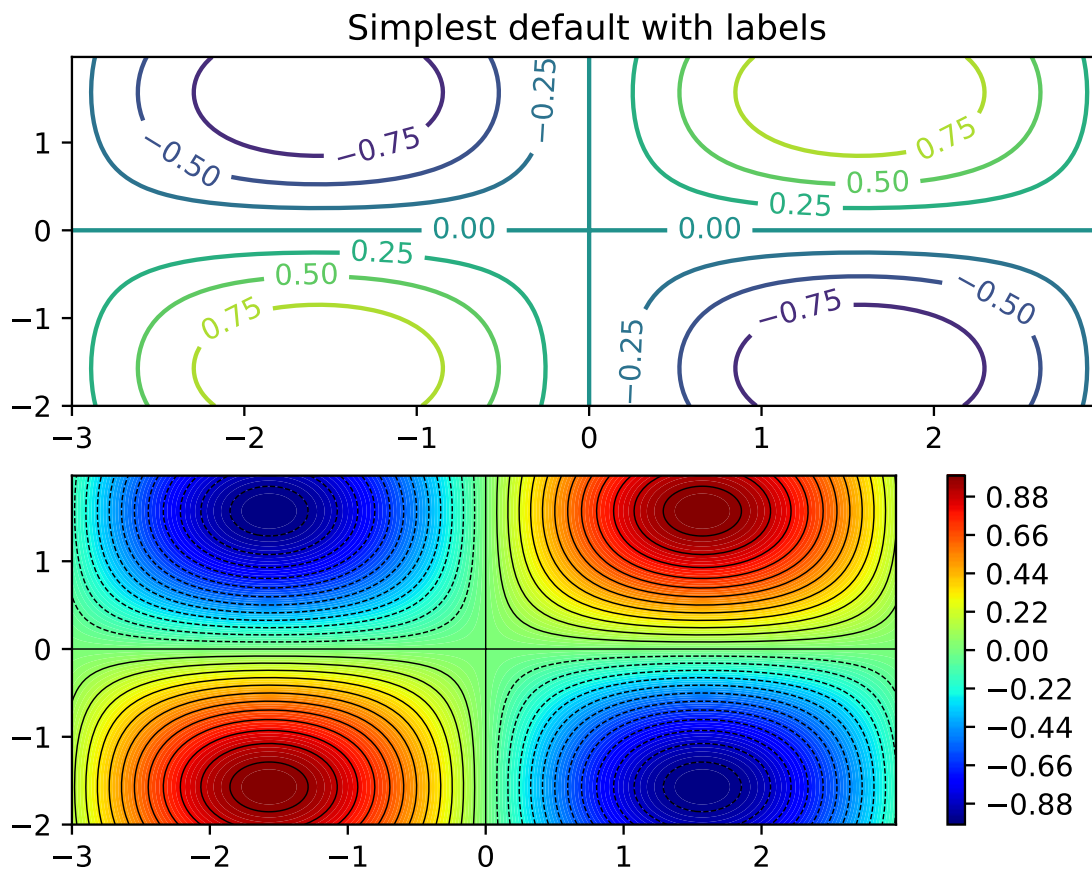
1 import matplotlib
2 import numpy as np
3 import matplotlib.cm as cm

```

```

4 import matplotlib.pyplot as plt
5
6 delta = 0.025
7
8 # Создаём оси координат
9 x = np.arange(-3.0, 3.0, delta)
10 y = np.arange(-2.0, 2.0, delta)
11
12 # Создаём сетку
13 X, Y = np.meshgrid(x, y)
14
15 # Задаём функцию на сетке
16 Z = np.sin(X)*np.sin(Y)
17
18 # Первый график - рисуем контурные линии
19 plt.subplot(2, 1, 1)
20 CS = plt.contour(X, Y, Z)
21 plt.clabel(CS, inline=1, fontsize=10)
22 plt.title('Simplest default with labels')
23
24 plt.subplot(2, 1, 2)
25 # Во втором графике рисуем контурные линии - 25 градаций
26 plt.contour(X, Y, Z, 25, linewidths=0.5, colors='k')
27
28 # Рисуем фон - 100 цветовых градаций
29 # cmap=plt.cm.jet - палитра
30 plt.contourf(X, Y, Z, 100, cmap=plt.cm.jet)
31 plt.colorbar()
32
33 plt.savefig("contours.pdf", bbox_inches='tight')
34
35 plt.show()

```



3.4 Подписи для функций

```

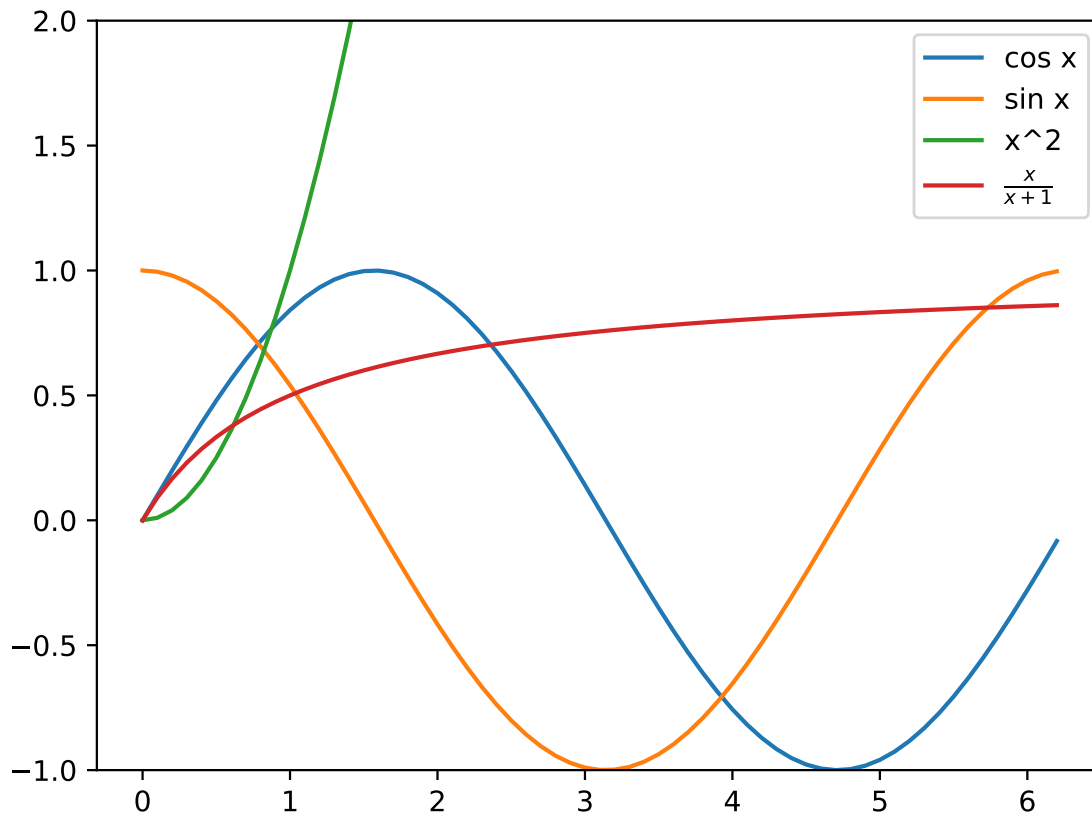
1 from matplotlib import pyplot as plt
2 import numpy as np
3
4 # Сетка
5 x = np.arange(0, np.pi * 2, 0.1)
6
7 # Четыре функции
8 y = np.sin(x)
9 y2 = np.cos(x)
10 y3 = x * x
11 y4 = x / (x + 1)
12
13 # Получаем класс axis для ограничения
14 # вертикальной оси координат
15 ax = plt.subplot()
16
17 # Устанавливаем пределы для значений по y
18 ax.set_ylim(-1, 2)
19
20 # Рисуем графики
21 plt.plot(x, y, label = 'cos x')
22 plt.plot(x, y2, label = 'sin x')
23 plt.plot(x, y3, label = 'x^2')
24 plt.plot(x, y4, label = '$\\frac{x}{x+1}$')
25 # Символ \ приходится дублировать.

```

```

26 # Если хотим писать как в системе LaTeX, то есть без дублирования,
27 # то нужно добавить символ r перед строкой.
28 # Получится label =  $r'\frac{x}{x+1}$ 
29
30 # Рисуем подписи
31 plt.legend()
32
33 plt.savefig("subscriptions.pdf", bbox_inches='tight')
34
35 plt.show()

```



3.5 Трёхмерные графики

```

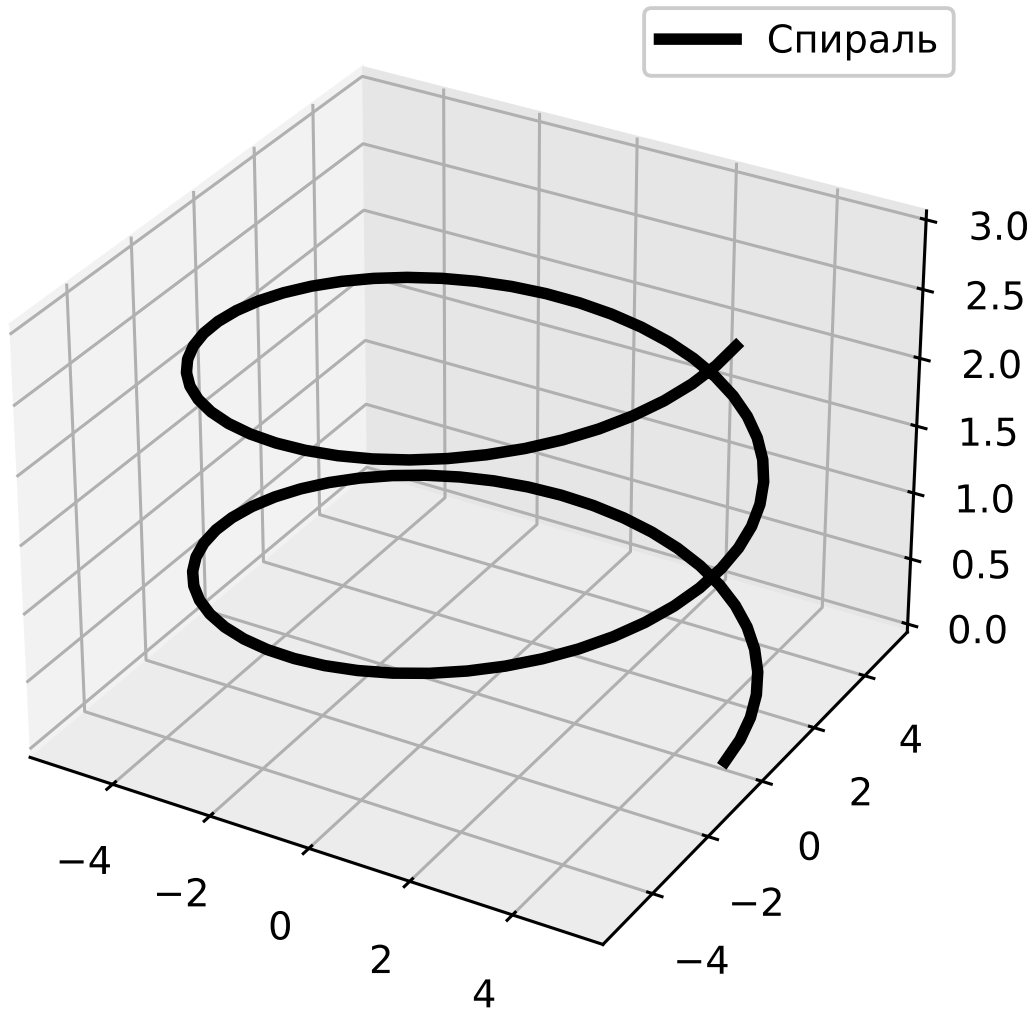
1 # Axes3D нужен для трёхмерных графиков
2 from mpl_toolkits.mplot3d import Axes3D
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6
7 fig = plt.figure()
8
9 # Для получения класса, описывающего трёхмерные
10 # координатные оси нужно передать параметр
11 # projection='3d' в метод add_subplot() класса Figure
12 ax = fig.add_subplot(projection='3d')
13
14 phi = np.linspace(0, 4 * np.pi, num=100)
15 r = 5
16 h = np.linspace(0, 3, num=100)

```

```

17
18 x = r * np.cos(phi)
19 y = r * np.sin(phi)
20 z = h
21
22 plt.plot(x, y, z, label='Спираль', linewidth=3., color='k')
23 plt.legend()
24
25 plt.savefig('3d.eps', bbox_inches='tight')
26
27 plt.show()

```



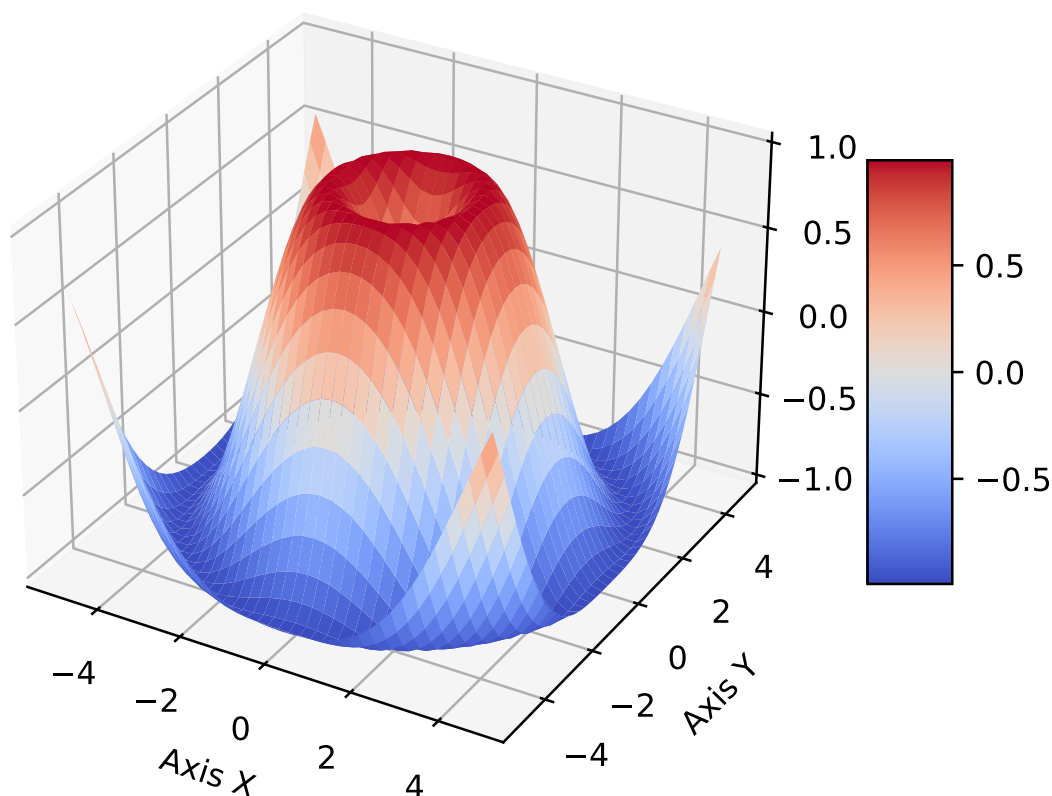
3.6 Рисование поверхностей

```

1 # Axes3D нужен для трёхмерных графиков
2 from mpl_toolkits.mplot3d import Axes3D
3 import matplotlib.pyplot as plt
4 from matplotlib import cm
5 import numpy as np
6
7
8 fig = plt.figure()

```

```
9
10 # Для получения класса, описывающего трёхмерные
11 # координатные оси нужно передать параметр
12 # projection='3d' в метод add_subplot() класса Figure
13 ax = fig.add_subplot(projection='3d')
14
15 # Создаём массивы данных.
16 X = np.arange(-5, 5, 0.25)
17 Y = np.arange(-5, 5, 0.25)
18 X, Y = np.meshgrid(X, Y)
19 R = np.sqrt(X**2 + Y**2)
20 Z = np.sin(R)
21
22 # Рисуем поверхность
23 surf = ax.plot_surface(X, Y, Z, cmap=cm.coolwarm)
24
25 # Устанавливаем масштаб по оси z
26 ax.set_zlim(-1.01, 1.01)
27
28 # Ставим подписи к осям
29 ax.set_xlabel('Axis X')
30 ax.set_ylabel('Axis Y')
31 ax.set_zlabel('Axis Z')
32
33 # Рисуем colorbar
34 fig.colorbar(surf, shrink=0.5, aspect=5)
35
36 # Параметр bbox_inches='tight' обрезает лишние поля
37 fig.savefig('surface2.pdf', bbox_inches='tight')
38
39 plt.show()
```



3.7 Рисование нескольких поверхностей на одном рисунке

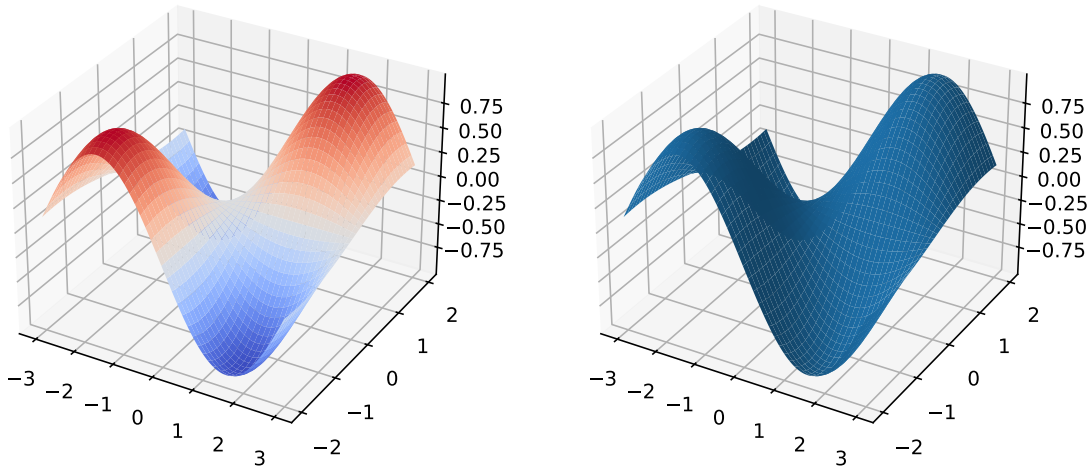
```

1 import numpy as np
2 from matplotlib import pyplot as plt
3
4 # Нужно для трёхмерных графиков
5 from mpl_toolkits.mplot3d import Axes3D
6 from matplotlib import cm
7
8 delta = 0.025
9
10 # Создаём сетку
11 x = np.arange(-3.0, 3.0, delta)
12 y = np.arange(-2.0, 2.0, delta)
13
14 X, Y = np.meshgrid(x, y)
15
16 # Задаём значения функции на сетке
17 Z = np.sin(X)*np.sin(Y)
18
19 fig = plt.figure(figsize = plt.figaspect(0.5))
20
21 # Добавляем правый график
22 # В отличие от двумерных графиков, которые описываются классом
23 # Axes, трёхмерные описываются классом Axes3D
24 # Для того, чтобы получить его экземпляр, нужно передать
25 # параметр projection='3d' в функцию add_subplot()
```

```

26 ax = fig.add_subplot(1, 2, 1, projection='3d')
27 ax.plot_surface(X, Y, Z, cmap=cm.coolwarm)
28
29 # Добавляем левый график
30 ax = fig.add_subplot(1, 2, 2, projection='3d')
31 ax.plot_surface(X, Y, Z)
32
33 # Параметр bbox_inches='tight' обрезает лишние поля
34 plt.savefig("surface.pdf", bbox_inches='tight')
35
36 plt.show()

```



3.8 Мультики!!!

```

1 import matplotlib as mpl
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import matplotlib.animation as animation
5
6
7 T = 100
8
9 # Выбираем encoder
10 # В данном случае FFMpeg
11 # Для этого он должен быть установлен в системе
12 #FFMpegWriter = animation.writers['ffmpeg']
13
14 # Если FFMpeg находится в нестандартной папке, то нужно прописать к нему путь.
15 #plt.rcParams['animation.ffmpeg_path'] = r'C:\ffmpeg\bin\ffmpeg.exe'
16
17 # fps - число кадров в секунду
18 #writer = FFMpegWriter(fps=25)
19
20 writer = animation.FFMpegWriter(fps=25)
21 #writer = animation.ImageMagickWriter(fps=25)
22
23 # размер шрифта для подписей

```

```

24 mpl.rcParams['legend.fontsize'] = 10
25
26 # Нам понадобится класс, описывающий рисунок
27 fig = plt.figure()
28
29 # Настройки encoder'a
30 # 200 - dpi (число точек на дюйм)
31 writer.setup(fig, "video.avi" , 200)
32 #writer.setup(fig, "video.gif" , 200)
33
34 # Создаём сетку
35 x = np.arange(0, np.pi * 5, 0.1)
36
37 for t in range(0, T):
38     # Выводим номер кадра на экран для удобства
39     print('Drawing frame %d...' % (t))
40
41     # Заводим функцию на сетке
42     y = np.sin(x - 2 * np.pi * t / T)
43
44     # Рисуем
45     plt.plot(x, y)
46     plt.title("T = %f" % (t))
47
48     # Сохраняем кадр
49     writer.grab_frame()
50
51     # Очищаем рисунок для следующей итерации
52     plt.clf()

```

Список литературы

- [1] Бизли Д. Python. Подробный справочник. СПб: Символ-Плюс. 2010.
- [2] <https://www.python.org>
- [3] <https://www.anaconda.com/distribution>